PeakForcast : An autonomic elasticity management model based on machine learning to absorb traffic surge in a cloud environment.

Daniel Rene Fouomene Pewo, *info@fouomene.com*

Abstract—This paper presents an approach to autonomic elasticity management in the cloud. Autonomic elasticity management in the cloud is the ability to automatically and dynamically control the supply of resources according to traffic fluctuations. The interest of the approach presented here lies, on one hand, on a process of continuous learning of traffic behavior, whose knowledge improves the detection of future traffic surges, and on the other hand, in a simultaneous allocation mecanism of resources during the process of auto-scaling cloud resources, with the aim of absorbing detected traffic surges. Autonomic computing is generally represented in elastic cloud-based systems by an elasticity controller with an autonomic MAPE-K control loop (Monitor, Analyze, Plan, Execute and Knowledge). In order to increase and maintain the reliability of the forecasting model implemented for the Analysis phase (Analyze), a machine learning phase (Learn) has been incorporated to extend the control loop MAPE-K to a MAPE-KL (Monitor, Analyze, Plan, Execute, Knowledge and Learn) loop. A supervised machine learning method is used with the implementation of a regression algorithm, in order to permanently fine tune the traffic forecasting model. The Actor model of concurrent programming is used to ensure simultaneous auto-scaling resources. In this model the parallelism of execution is expressed explicitly. We have evaluated the approach using MediaWiki and web application traffic traces that were collected on the Japanese version of Wikipedia some days before and after the tsunami on March 11th, 2011, in an interval where the traffic surge has been observed. The experimental results confirm that our prototype elastic middleware solution(PEAKFORECAST MIDDLEWARE), can provide instantaneous elasticity of resources during traffic surges.

Index Terms—Cloud, Traffic surge, IaaS, Elasticity, MAPE-K, Traffic Forecasting.

1 INTRODUCTION

AUTONOMIC elasticity management in the cloud is the ability to automatically and dynamically control the supply of resources according to traffic fluctuations. Autonomic computing is generally represented in elastic cloud-based systems by an elasticity controller with an autonomic MAPE-K control loop [1] (Monitor, Analyze, Plan, Execute and Knowledge). The MAPE-K loop model can be applied to implement a cloud web application system which knows its state and reacts to its changes [3]. Therefore, the majority of elasticity mechanisms are based on the MAPE-K loop [2], [3], [4], [5], [6].

Abrupt changes in load, or traffic surge, are all too common in today's data centers. Important events, such as the September 11 attacks [7], [8], earthquakes or other natural disasters [9], Slashdot effects [10], Black Friday shopping [11], or sporting events, such as the Super Bowl [12] or the Soccer World Cup [13], are common causes of load spikes for website traffic. Big news breaks such as the Japan tsunami or the death of Osama bin Laden or Michael Jackson are common causes the spike in Yahoo's traffic [16]. Service outages [14] or server failures [15] can also result in abrupt changes in load caused by a sharp drop in capacity. One recent Facebook application experienced 10 times increase in the number of users from 25,000 to 250,000 in just three days with up to 20,000 new registrations per hour in peak times [17]. Such typically business-critical systems must satisfy certain level of service level agreements (SLA), e.g., upper bounds on user perceived response time. Otherwise, unexpected loads cause a poor service level that frustrate end users [18]. Amazon reported a loss of 245 million dollars for an increase of 100ms in response time [19]. While some of the above events are predictable, most of them cannot be predicted in advance, like the Slashdot effect, which occur when a distributed system is overloaded by a huge number of requests, potentially leading to its unavailability.

There is a lot of prior works that deal with dynamic resource management, to efficiently utilize elasticity of clouds. These works can be classified into predictive ([20], [21], [22], [23], [24], [25]), reactive ([26], [27], [28], [29]) and mixed ([30], [31], [32], [33], [34], [35], [36]) approaches. While these approaches can handle gradual changes in load, they cannot handle abrupt changes, especially traffic surge that occur almost instantaneously.

In this context, it is legitimate to raise the following research questions: *Is it possible to anticipate the effects of a traffic surge by protecting the targeted system from unavailability?*

In this paper, in order to address this challenge, we propose :

- An approach for the autonomic management of elasticity taking the form of an autonomic manager named PEAKFORCAST, of MAPE-KL type (Monitor, Analyze, Plan, Execute and Learn) where a new phase of Learn has been incorporated to extend the basic MAPE-K control loop. We describe the PeakForecast's Architecture and present the different conceptual bricks constituting it. The interest of this

approach lies, on one hand, on process of continuous learning of traffic behavior, whose knowledge improves the detection of future traffic surges, and on the other hand, in a simultaneous allocation mecanism of resources during the process of auto-scaling cloud resources, with the aim of absorbing the future traffic surges.

- An autonomous system supervision mechanism for an instantaneous elasticity of cloud resources : this supervision mechanism (the PeakForecast's Supervisor) coordinates the activation of the different elements of PeakForecast'Architecture in order to quickly reach a decision of optimal preference.
- A prototype of an autonomous system for an instantaneous elasticity of Cloud resources called PEAKFORECAST MIDDLEWARE which implements all the proposed concepts (the PeakForecast's Architecture and PeakForecast's Supervisor), capable of anticipating sudden peaks in load in order to maintain the quality of service, this by automatically and intelligently adjusting the capacity or resources of the Cloud on demand (auto-scaling based on demand). By using its monitoring service, the prototype can trigger the necessary actions in order to adjust the capacity up or down, depending on metrics (server utilization rate and network I/O or HTTP traffic, for example)

Our mix approach consists, from a trace of queries received in the last seconds, minutes or hours of : *i)* Detecting potential traffic surges, *ii)* Forecasting the upcoming traffic and, *iii)* Estimating the number of resources required to absorb the remaining traffic to come, *iv)* Auto–scaling resources by quickly automatically adding resources to absorb traffic surges.

The rest of the paper is organized as follows. Section 2 we discuss related works in this domain. Section 3 describes Peakforecast's approach. Section 4 is entirely devoted to the experimentation of the model, where we evaluate our approach, by using the WEB Mediawiki application and the traffic traces that were collected on the Japanese version of the site Wikipedia the days before and following the Tsunami of March 10, 2011.

2 STATE OF ART

For a comprehensive study regarding the state of the art of elasticity in the cloud, We highlight the most prominent elasticity related solutions regarding (1) reactive approaches, (2) predictive approaches, and (3) mixed approaches. For a more detail review, refer to [60], [61], [62], [63]; The studies focused on each MAPE-K phase. The Monitoring(M) phase is responsible for collecting information about the metrics (e.g., CPU, RAM, network traffic, etc...). These different sets of monitoring parameters are stored in a Knowledge(K) base for use by other phases. The knowledge base in the MAPE-K loop model shares data between the actual MAPE phases. In the Analysis(A) phase, the analysis is carried out with reactive or proactive policies. For scaling decision-making in the Planning(P) phase, different techniques such as rule-based decisions are used for resource estimation. In the Execution(E) phase, scaling is done with horizontal and vertical methods which work with replication-based and resizing-based techniques, respectively.

Reactive approaches such as threshold-based rules or policies are the most popular technique for automatic scaling in cloud computing (commercial systems such as provided by Amazon Web Services (AWS) [64], RightScale [65], Microsoft [66]), presumaly because of their apparent simplicity. The Amazon Web Services (AWS) [64] cloud platform offers static threshold scaling through specific metrics such as the CPU utilization. Amazon then monitors these metrics and scales when the set threshold rules are breached. Amazon offers a few different techniques on how the scale performs such as a fixed change, adding/negating from the current machine count or using a percentage change for the machines. Microsoft Azure [66] offering of static threshold-based rules runs the choice of two metrics, CPU utilization and the number of work tasks in the message queue [43]. RightScale offer the same core use of static threshold-based rules as the other two providers mentioned. RightScale have extended static threshold-based rules with RightScale's auto-scaling algorithm [67]. It is a simple democratic voting process whereby, if a majority of the VMs agree that they should scale up or down, that action is taken; otherwise no action occurs. Each VM votes to scale up or down based on a set of rules. After each scaling action, there is a period called the resize calm time (equivalent to the inertia or cooldown time), where no action can be performed. It prevents the algorithm from continually allocating resources as new instances boot [2]. As RightScale's voting system is based on rules,it has the same disadvantage [2]: the algorithm is highly dependent on user-defined threshold values, and therefore, to the workload characteristics. This was the conclusion reached by Kupferman et al. [69] after comparing RightScale with other algorithms. Static threshold–based rules presented the most simple and intuitive approach for auto scaling. However, setting the appropriate thresholds is a very tricky task, and can lead to instability in the system. Moreover, static thresholds become invalid if the behavior of the application changes suddenly during peak traffic. Setting the correct threshold also presents a challenge as this is done manually which requires knowledge of the application. If the threshold is wrong there is the risk of oscillation in the scaling, underutilization of resources or over utilization [70]. In Reactive approaches, all of his work uses a MAPE cycle-based scaling mechanism with a purely reactive policy using the scaling indicator of the response time. They focuse on both Analysis and Planning phases (with more focus on the planning phase where different techniques such as rule-based decisions are used for resource estimation).

Predictive approaches use heuristics and analytical techniques together with historical data to predict future demand and proactively allocate resources. A number of different methods are used in the workload prediction. In [25], it develops a model–predictive algorithm for the workload prediction in which a second order autoregressive moving average method filter is used. PRESS [23] (Fig 1) developed a hybrid online resource demand prediction model that combines a Markov model and a fast Fourier transform–based technique. Previous prediction schemes either focus

on short–term prediction or need to assume cyclic workload patterns. Gmach et al. [71] used a Fourier transform-based scheme to perform offline extraction of long–term cyclic workload patterns. In comparison, PEAKFORECAST does not assume the workload is cyclic, and can predict resource demands for arbitrary workload patterns. Chen et al. [72] used sparse periodic auto-regression to perform load prediction. However, their approach is tailored towards long prediction intervals (e.g.,hours) and assumes that the repeating period is known in advance [23]. The experiments [23] have shown that auto-regression is computationally intensive, which makes it impractical for short-term online VM resource scaling. Saripalli et al. [73] use a two-step approach, using cubic spline interpolation combined with a hotspot detection algorithm for sudden spikes. Predicted values depend on the window width chosen. Hence, large amounts of available data are needed to produce confident predictions [20]. In Predictive approaches, all of his work focuse on the Analysis phase where they predicted the future of resource utilization.

Mixed approaches. Mixed or Hybrid approaches combine reactive and predictive approaches to determine when to acquire resources over short and long time scales respectively (e.g., [30]). In this category, some approaches use predictive techniques for releasing resources and reactive techniques for acquiring resources(e.g., [31], [32]). Laura R. et al [31] presented a Platform Insights is a hybrid elasticity controller employing both reactive rule–based and predictive model–based elasticity mechanisms together in a coordinated manner. However, Platform Insights not handle multiple QoS objectives at once and not incorporate an algorithm to detect change in workload mix. In comparison, PEAKFORECAST detects change in workload mix. From a trace of requests received in the last seconds, minutes or hours, PEAKFORECAST detects if the underlying system is facing a traffic surge or not.

Previous work has applied control theory [74], [75], [76] or reinforcement learning [77] to adaptively adjust resource allocations based on service level objectives (SLO) conformance feedback. However, those approaches often have parameters that need to be speciffed or tuned offine, and need some time to converge to the optimal(near-optimal) [23]. In comparison, PEAFFORECAST directly predicts resource allocation based on historical request time series.

Huber et al. [78] presented a self-adaptive resource management algorithm which leverages workload prediction and a performance model [79] that predicts application's performance under different configurations and workloads [80]. In comparison, PEAKFORECAST does not require any prior application knowledge.

Vertical vs. Horizontal scaling. In practice [61], existing solutions for auto-scaling enable horizontal scaling, i.e., acquiring or realizing node instances, while vertical scaling, i.e., increasing computing power of node instances, is not considered. It has been attributed to impossibility of changing the size of nodes at the hypervisors level [60] [81]. SmartScale [33] proposed an auto-scaling method that minimized resource usage costs for bag-of-tasks jobs. It used horizontal scaling which added or removed VMs and vertical scaling which expanded or reduced the size of a VM. However, it is still deficient for resource requirements of dynamic workloads because it lacks consideration of resource usage during execution of an application. PEAKFORECAST enables horizontal scaling.

It should be noted that the predictive models and the various theories mentioned above are mainly reserved for the scientific world. Indeed, industrial solutions prefer a reactive approach based on threshold-based rules. This is due to the fact that this type of solution is simpler to implement and more easily understandable by a non-scientific community that is struggling to apprehend theories that are too far from the industrial world. The main disadvantage of reactive techniques is that they do not anticipate unexpected changes in workload, and therefore, resources can not be provided in advance.

In order to handle traffic surges, the authors advocate either having spare servers that are always available (*i.e.*, over-provisioning), or finding a way to lower setup times. Which considers that the architecture of the datacenter includes a *caching* tier consisting of machines that are always available. They temporarily use the resources of these machines to process requests as long as new servers are being deployed in the application tier. The elastic part is limited to the application tier, which is the largest consumer of CPU resources. When the load is high, needs to be coupled with techniques like those in [82], [83], [84], [85] to minimize the damage caused by load spikes. However, by using PEAKFORECAST, we do not have to pay for any additional resources, which is not the case when over-provisioning via spare servers. Furthermore, we used mixed approaches (reactive and predictive), which allows us to manage traffic surges in less than 60 seconds for case of VMs or in less than 10 seconds for case of Containers, in order to respond with the best reactivity as possible to user demands and maintaining the quality of service online site continuously.

There is also work, such as [85], [86], which addresses elastic control for multi-tier application services that allocate and release resources in discrete units, such as virtual server instances of pre-determined sizes. It focuses on an elastic control of the storage tier, in which adding or removing a storage node or *brick* requires rebalancing stored data across the nodes, and are not the focus of our paper.

TABLE 1
Summary of the PEAKFORCAST comparison in the same problem

References	Approa -ches		Methods Scaling		Focused Phase MAPE-KL					Limitations
	Reactive	Predictive	Vertical	horizontal	Monitoring	Analysis	Planning	Execution	Learning	
[64](2017)	x			x	x		x	x		Reactive approaches presented the most simple and intuitive approach for auto scaling. However, setting the appropriate thresholds is a very tricky task, and can lead to instability in the system. Moreover, static thresholds become invalid if the behavior of the application changes suddenly during peak traffic.
[65], [67](2017)	x			x	x		x	x		
[66](2017)	x			x	x		x	x		
[41](2010)	x			x	x		x	x		
[22](2011)		x		x	x	x		x		Previous prediction schemes either focus on short-term prediction or need to assume cyclic workload patterns. In comparison, PeakForecast does not assume the workload is cyclic, and can predict resource demands for arbitrary workload patterns. PRESS [23] does not manage the monitoring or the execution of actions and is only interested in the sizing decision (see Figure 1); Observations and adaptations are respectively provided and executed by the system(i.e. IaaS). In comparison, the second component allows PEAKFORECAST to monitor HTTP traffic, detecting traffic surges and reacting accordingly by consuming the services provided by the first component.
[23](2010)		x		x	x	x		x		Their approach is tailored towards long prediction intervals (e.g, hours) and assumes that the repeating period is known in advance.
[73](2011) [76](2014)		x		x	x	x		x		Predicted values depend on the window width chosen. Hence, large amounts of available data are needed to produce condent predictions.
[74], [75], [76](2009)		x		x	x	x		x		Those approaches often have parameters that need to be specified or tuned offine, and need some time to converge to the optimal(near-optimal) decisions. In comparison, PeakForecast directly predicts resource allocation based on historical request time series
[80](2013)		x		x		x				Presented a self-adaptive resource management algorithm which leverages workload prediction and a performance model that predicts application's performance under different configurations and workloads (see Fig 2). In contrast, PeakForecast does not require any prior application knowledge. AGILE [80] is not intelligent enough because it does not have a traffic surge detection mechanism. In comparison, PEAKFORECAST can to detect when we are facing a traffic surge (see Section 4.1.1) through actor Analyse.
[31](2013)	x	x		x	x	x		x		Platform Insights not handle multiple QoS objectives at once and not incorporate an algorithm to detect change in workload mix.In comparison,PEAKFORECAST detects change in workload mix.
[33](2013)	x	x	x	x	x	x		x		it is still deficient for resource requirements of dynamic workloads because it lacks consideration of resource usage during execution of an application.
[61](2014)	x	x	x	x	x	x	x	x		not comparison with other auto-scaling approaches in controlled experiments.
Our Proposed	X	X		X	X	X	X	X	X	

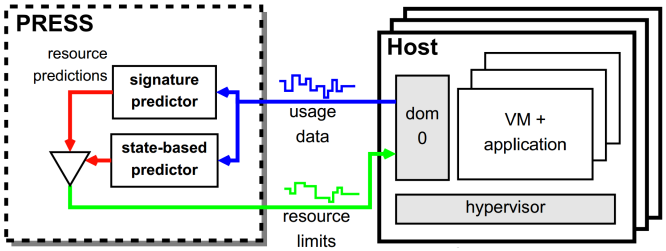

Fig. 1. Overall architecture of the PRESS system. The host system runs applications in virtual machines (VMs), and monitors their resource usage. This data is fed to PRESS, which makes predictions of future application resource needs, which are sent back to the host system, which scales the resource-limits on each VM accordingly [23]

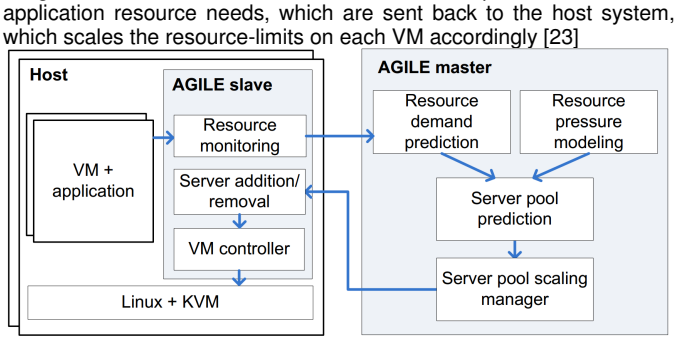

Fig. 2. The overall structure of the AGILE system. The AGILE slave continuously monitors the resource usage of different servers running inside local VMs. The AGILE master collects the monitor data to predict future resource demands. Based on the resource demand prediction result and the resource pressure model, the AGILE master invokes the server pool manager to add or remove servers. [80]

3 The PeakForecast Approach

In this section we present our contributions, which contribution consists of our Generic architecture for autonomic manager system in cloud PeakForcast's architechture (sub-section 3.1), PeakForcast's supervisor (sub-section 3.2) and PeakForcast Middelware (sub-section 3.3).

3.1 The PeakForecast's Architecture

This section presents the PeakForecast's Architecture of our model. This architecture has two main conceptual bricks : the PeakForcast's Client whose role is to allow to monitor HTTP traffic, detecting traffic surges, and the PeakForcast's Server which exposes web services to administrate the resources (VMs or containers) of the virtual web infrastructure. Its web services are consumed by PeakForcast Client to react accordingly to traffic surges.

3.1.1 PeakForcast's Client

It contains one essential component Spirals, wich implements a MAPE-KL loop where 'L'(Learn, Learning Process) is a new phase that we introduced in the IBM MAPE-K loop and composed of five actors : Monitor, Learn, Analyse, Plan and Execute, communicating with each others by exchanging messages. In Knowledge base, we save the traffic http collected in real time, to exploit during the Learning Process.

- **Monitor :** collects traffic information per time interval in real time and save in *Knowledge* base. The number of requests collected by the actor Monitor are sent as a message to the actor Analyse.

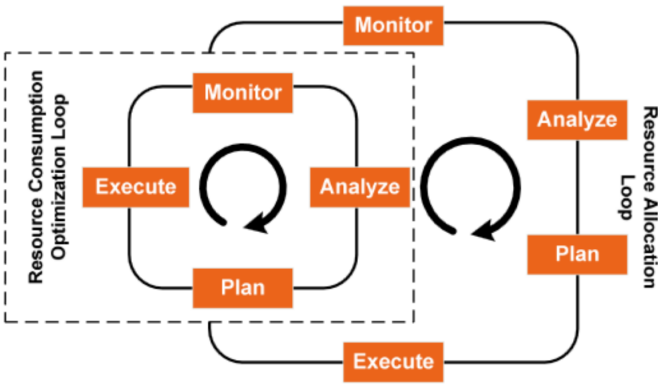

Fig. 3. Control loops of the SmartRod solution proposed by. [67]

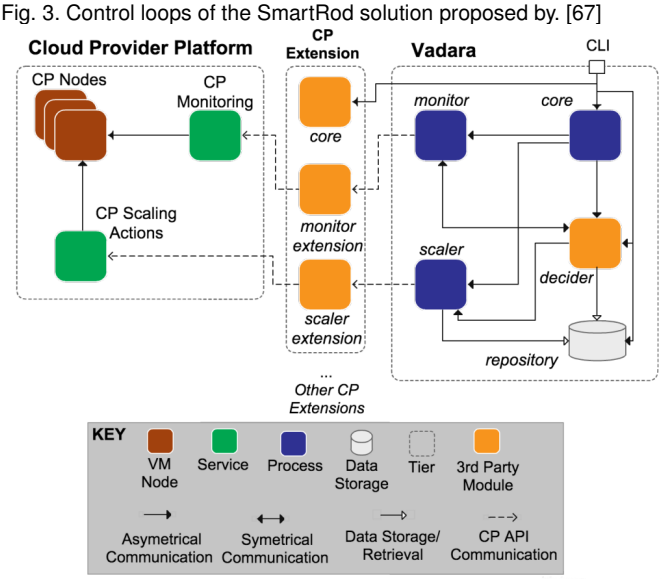

Fig. 4. Vadara's Architecture by. [20]

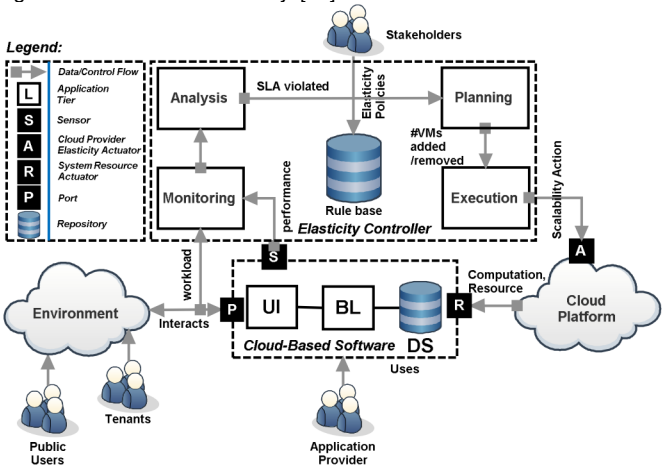

Fig. 5. High-level view of elastic software. Figure gives an overview of the solution space : ascalable cloudbased application hosted on nodes obtained from a provider based on a pay-as-you-go lease. [61]

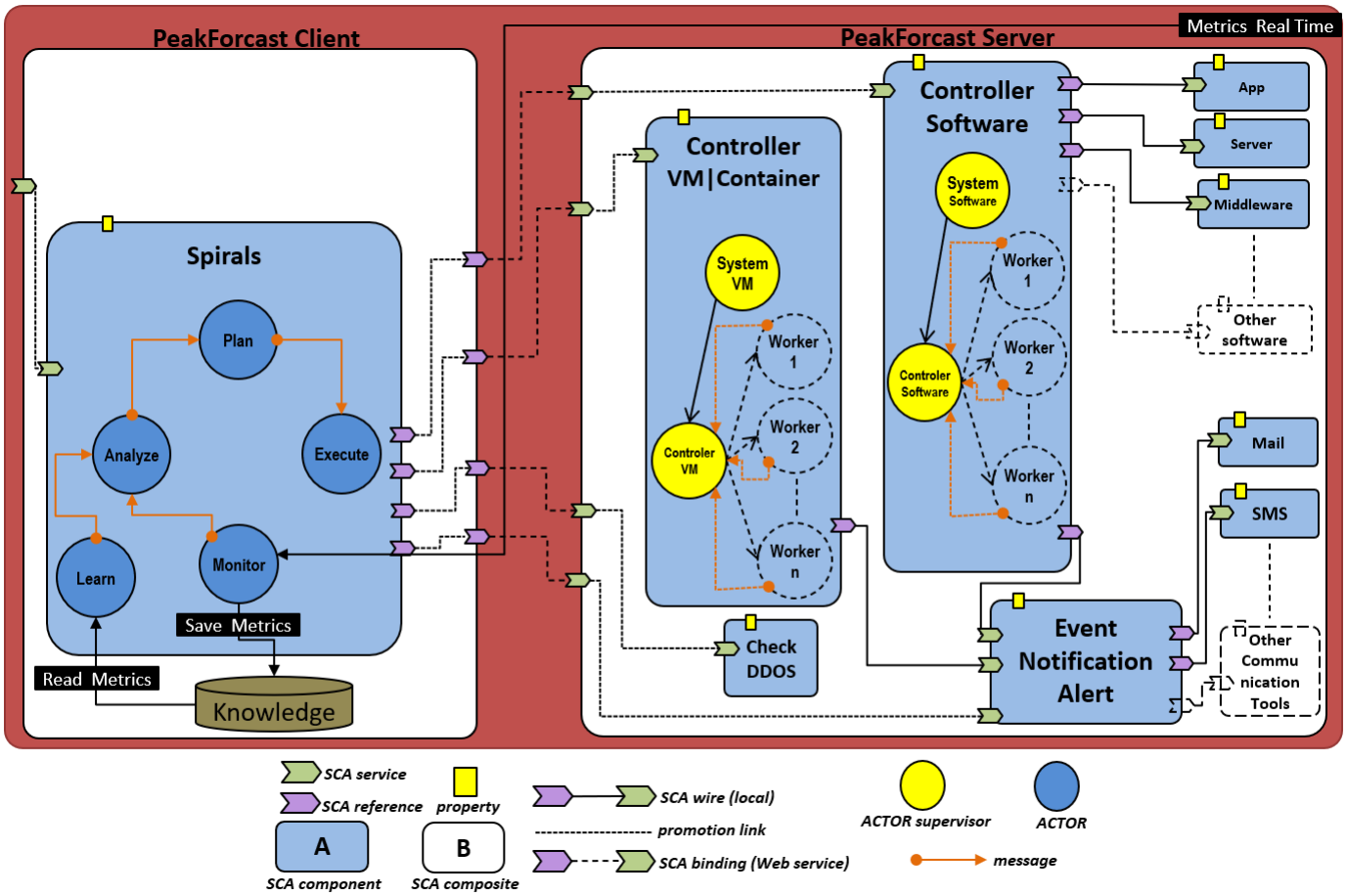

Fig. 6. The PeakForcast Generic architecture

- **Analyse :** analyzes this number of requests in *i)* Detecting Potential Traffic Surges, *ii)* After detecting the traffic surge, forecasting the Upcoming Traffic by using a forecasting model, *iii)* Estimating the number resources required to absorb the remaining traffic to come. Finally, the actor Analyse sends this number of resources (VMs or Containers) as a message to the actor *Plan*.
- **Plan :** based on the latency of VMs in use, determines the exact number of VM or container instances to deploy in order to absorb the traffic surge, verifies if the virtual web infrastructure is facing a *Distributed Deny of Service* (DDoS), and then sends this number to the actor Execute. To provide this decision-making capacity, the actor *Plan* uses active rules or ECA (*Event-Condition-Action*) rules.
- **Execute :** takes care of providing the requested VMs or container instances, allowing the virtual web infrastructure to anticipate the traffic surge ahead.
- **Learn :** based on Learning Supervised [38] as a machine learning technique, allows calculation (Learning Process) to part of a historic request (*e.g.*, 30 numbers of requests received) save in *Knowledge* base , to determine the best *smoothing constant* α and to send α as a message to the actor Analyse in order to increase and maintain the reliability of prediction model ([56]). In order to manage the positive faults, the actor Learn which allows us to part of a historical request (eg 30 numbers of requests received), to determine at each moment, the best smoothing constant of the moment that minimizes the sum of the squares of the last third of the errors. Thus, the value of the smoothing constant is dynamically updated in order to increase and maintain the reliability of prediction model.
- **Knowledge base :** we save the traffic http collected in real time, to exploit during the Learning Process.

3.1.2 *PeakForcast's Server*

It contains four essential component ControllerSoftware, Controller VM—Container, CheckDDOS and EventNotificationAlert.

- **ControllerSoftware :** provides services to *install, uninstall, reconfig, stop, start, restart* a component on one or more VMs. Here, one component may be a server or an application. In case the service must be running on multiple VMs, the operation is performed by the actors Worker"n" `to operate in parallel` and coordinated by the supervisor actor ControllerSoftware.
- **Controller VM—Container :** provides services to *start, restart, pause, resume, clone, create, delete* VMs and Containers. All these services are applied to one or more VMs and Containers.

- **CheckDDOS :** allows the component Spirals, to check if a server is under distributed denial-of-service (DDoS) before adding resources within a virtual web infrastructure.
- **EventNotificationAlert :** provides services to send emails, tweets and SMS notifications or to alert to administrators of the virtual web infrastructure. The administrators can receive a notification message even if they are on the move to keep being informed and up–to–date about the condition of virtual web infrastructure.

An overview detail of the PeakForcast's Architecture is given in the Fig 6

3.2 The PeakForcast supervision mechanism

PeakForcast's supervisor is based on our extended MAPE-KL model of autonomous systems. The objective of this algorithm is to answer three questions : When should you scale up? How much resources are needed? How do I go to scale?

PeakForcast's supervisor coordinates the activation of the various components of the architecture in order to quickly arrive at an optimal preference decision. To provide this decision-making capacity, the supervision algorithm uses active rules or ECA (*Event-Condition-Action*) rules, widely studied in the field of active databases.

The supervision mechanism is based on the following environment variables :

- *infrastructureIsUp* (iIU) allows to check if infrastructure is running, which is initial to true.
- *intervalTime* (iT) represents the time interval the actor Monitor will use to collect requests, which is initial to every 30 seconds.
- *intervalHistoryRequests* (iHR) represents the interval of the last analysis requests that the actor Learn will use during the automatic learning process, which is initial to 100 requests .
- *constantSmoothing* (cS) represents the smoothing constant of the predictive model by the Simple Exponential Smoothing method, which is initial to 0.9 for example based on requests history.
- *numberRequestCurrent* (nR) represents the number of request collected after an *intervalTime* time recorded in the *numberRequestCurrent* variable.
- *numberRequestPrevious* (nRP) contains the previous value of *numberRequestCurrent*. Initially they have the same value).
- *numberVMEstimate* (nVME) represents the number of VMs or Containers estimate by actor Analyse.
- *numberVMEstimateFinal* (nVMEF) represents the number of VMs or Containers determines by actor Plan.

3.2.1 PeakForcast Supervision Main Procedure

Main Procedure describes a control loop mechanism MAPE-KL that runs as long as the virtual infrastructure is active.

***Line 2* (Monitor)** corresponds to the collection of the number of requests per time interval (for example one second, 15 seconds, 30 seconds, 1 minute) of the virtual infrastructure in real time.

1 Main Procedure

Require: INITIALIZATIONENVIRONMENTVARIABLES()
1: **while** iIU **do**
2: $nRC \leftarrow$MONITOR(iT)
3: $cS \leftarrow$ LEARN(iHR)
4: $nVME \leftarrow$ANALYZE(nRC, nPR, cS)
5: $nVMEF \leftarrow$PLAN($nVME$)
6: EXECUTE($nVMEF$)
7: $nRP \leftarrow$nRC
8: **end while**

***Line 3* (Learn)** corresponds to the analysis (automatic learning process) of the numbers of requests collected in order to improve the predictions or resource estimates needed to absorb the peak of future traffic, based on Learning Supervised [38] as a machine learning technique.

***Line 4* (Anlyze)** corresponds to the analysis of the number of requests as they are collected; When an upcoming peak is detected, an estimate of the number of resources (VMs or Docker containers) needed to absorb the upcoming traffic peak is returned.

***Line 5* (Plan)** corresponds to the decision of the exact number of resources (VMs or Docker containers) to add to absorb the peak of traffic depending on the customer profile of the virtual infrastructure and resources in use.

***Line 6* (Execute)** corresponds to the execution of adding or removing a number of resources (VMs or Docker containers).

The functions Monitor, Learn, Analyze, Plan and Execution are described respectively below.

The interactions by exchange of messages between the actors are represented in the Figure 7.

3.2.2 MONITOR

The function monitor describes a sequence of collects traffic information per time interval, save in the database, and which are returned.

1: **function** MONITOR(iT)
2: Get nR After iT
3: Save nR in in *KNOWLEDGE* base.
4: **return** nR
5: **end function**

3.2.3 LEARN

The function describes a sequence that allows , to part of a historic request (*e.g.*, 30 numbers of requests received), to determine (learning process), the best *smoothing constant* α and and return it.

1: **function** LEARN(iHR)
2: With the iHR, calculate the best cS of simple exponential smoothing method for prediction
3: **return** cS
4: **end function**

3.2.4 ANALYZE

The function analyse describes a sequence that corresponds to the analysis of the number of requests as they are collected; When an upcoming peak is detected, an estimate of the number of resources (VMs or Docker containers) needed to absorb the upcoming traffic peak and return it.

The actor **Analyse** analyzes this number of requests in, detecting Potential Traffic Surges, after detecting the traffic surge, forecasting the Upcoming Traffic by using a forecasting model, estimating the number resources required to absorb the remaining traffic to come.

The function analyse is based on the following environment variables :

- *surgeCoefficientThreshold* (sCT) represents the highest C_s, the more intense the traffic surge, which is initial to 2.
- *numberDetectedPeak* (nDP) represents the number of detected traffic surges with an added VMs or Containers action, which is initial to 0.
- *listNumberRequestBeforePeak* (lNRBP) represents the list containing the values of the number of requests collected before the detected traffic peaks, which is initial to Null).
- *listNumberVMsUp* (lNVMUp) represents the list of VMs or Containers numbers added by the actor Execute, which is initial to Null.
- *scaleUp* (scaleUp) allows to trigger the action of adding the VMs or Containers, which is initial to false.
- *scaleDown* (scaleDown) allows to trigger the removal action of the VMs or Containers, which is initial to false.

```
function ANALYZE(nRC, nRP,cS)
    sC ← nRC/nRP
    if sC ≥ sCT then
        nDP ← (nDP + 1)
        lNRBP.add(nRP)
        With the simple exponential smoothing prediction method and the parameters nRC, cS, calculate the nRF
        With the nRF estimating the number of VMs nVME requirements to absorb traffic surge
        scaleUp ← true
        return nVME
    end if
    if lNRBP.isNotEmpty() then
        if lNRBP.get(nDP) ≥ nRC then
            nVMUp ← listNumberVMUp.get(nDP)
            scaleDown ← true
            return nVMUp
        end if
    end if
end function
```

3.2.5 PLAN

The function describes a sequence that corresponds to the decision of the exact number of resources (VMs or Docker containers) to add to absorb the peak of traffic depending on the customer profile of the virtual infrastructure and resources in use and return it.

```
function PLAN(nVME)
    if scaleDown = true then
        return nVMEF
    else
        With latency VMs running, the IAAS customer profil, nVME, determine the final number of VMs nVMEF
        return nVMEF
    end if
end function
```

3.2.6 EXECUTE

The function describes a sequence that corresponds to make available the requested VM or container instances, allowing the virtual web infrastructure to anticipate the traffic surge ahead.

```
function EXECUTE(nVMEF)
    if scaleDown = true then
        remove nVMEF VMs in infrastructure
        lNVMUp.remove(nDP)
        lNRBP.remove(nDP)
        nDP ← (nDP - 1)
        scaleDown ← false
    end if
    if scaleUp = true then
        add nVMEF VMs in infrastructure
        lNVMUp.add(nVMEF)
        scaleUp ← false
    end if
end function
```

3.3 PeakForecast Middleware

In this section, we provide a detailed overview (sub-section 3.3.1), the execution environment and operation (sub-section 3.3.2), the main conjectures concerning (sub-section 3.3.2) of PEAKFORECAST MIDDLEWARE.

3.3.1 PeakForecast Middleware Overview

PEAKFORECAST MIDDLEWARE is an elastic middleware solution, which implements all the proposed concepts (Peakforecast's Architechture and PeakForcast Supervision Mechanism), that we developed to automatically address the elasticity issues imposed by traffic surges not only by adding resources to absorb traffic surges, but also to release these resources once the load decreases.

PEAKFORECAST MIDDLEWARE is a distributed software system. PEAKFORECAST MIDDLEWARE therefore monitors the incoming traffic, makes predictions on the volume of this traffic and adjust the size of the infrastructure according to these predictions.

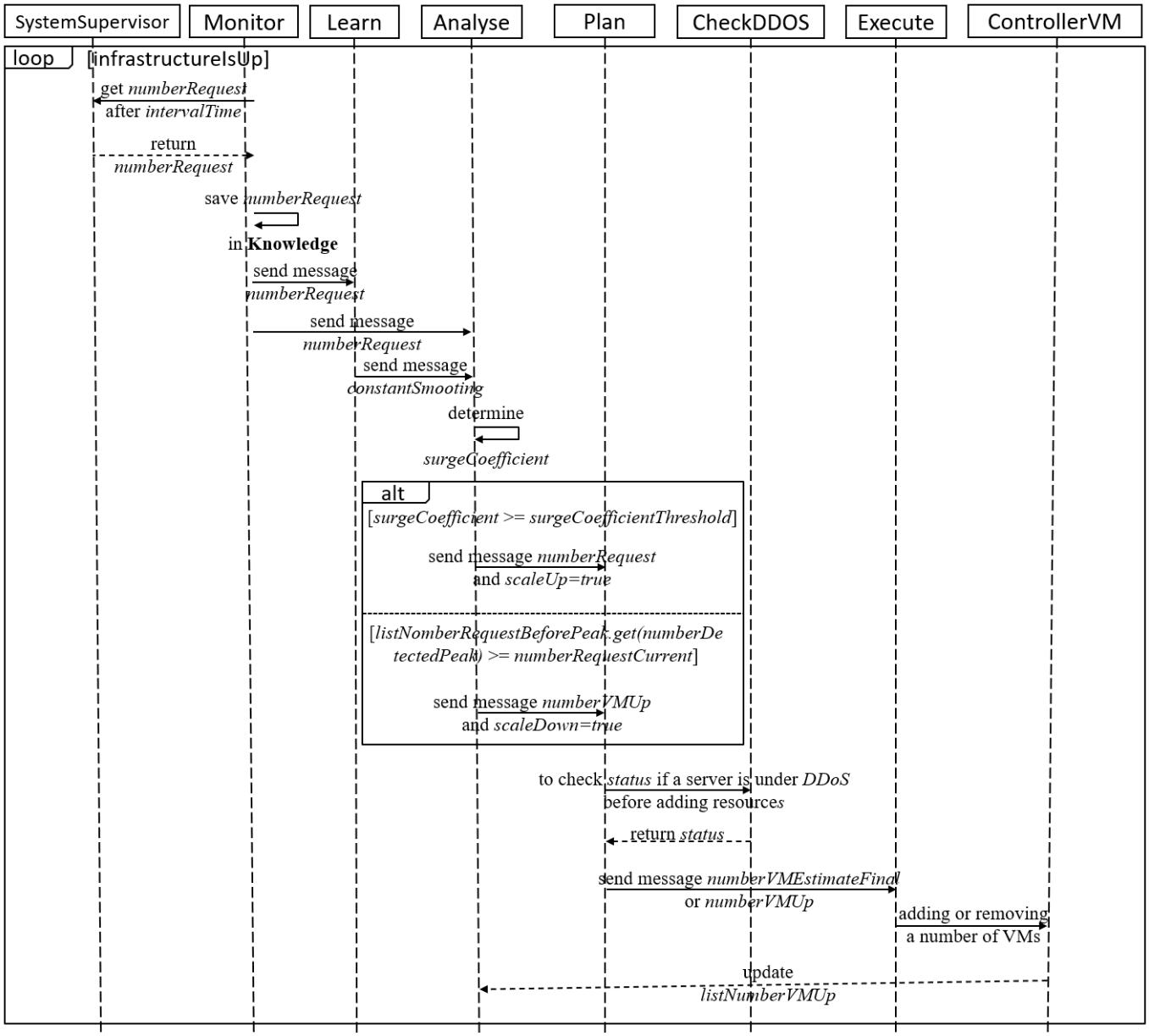

Fig. 7. Coordination between PEAKFORECAST's components.

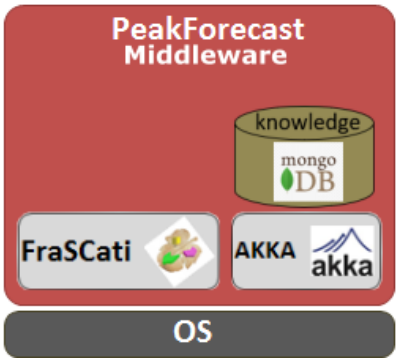

Fig. 8. PEAKFORECAST MIDDLEWARE.

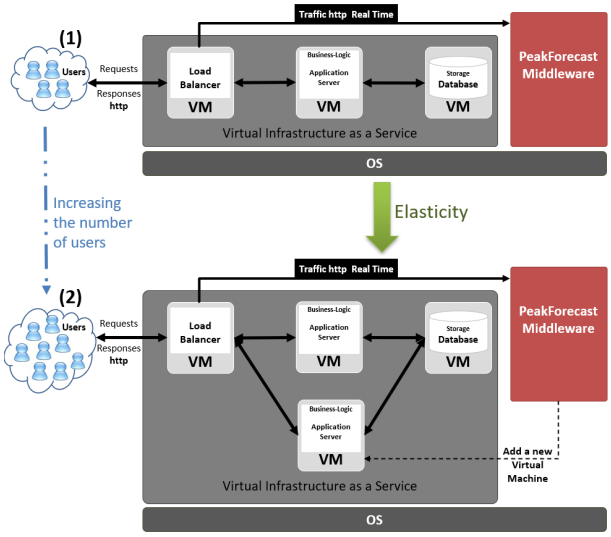

Fig. 9. PEAKFORECAST MIDDLEWARE elasticity operation for web application with a 3-tier architecture on a IaaS Infrastructure.

We use the FRASCATI [51][1] middleware platform, which enables the development and the execution of distributed SOA applications based on the SCA standard. FraSCAti offers a component-based approach to support the heterogeneous composition of different interface definition languages (WSDL, JAVA), implementation technologies (Java, Spring, EJB, BPEL, OSGI, Jython, Jruby, Xquery, Groovy , Velocity, Fscript, Beanshell), network communication protocols (Web Services, JMS, RPC, REST, RMI, UPnP). Beyond the support of standard SCA features, FRASCATI brings reflection to SCA—*i.e.*, introspection and reconfiguration capabilities, via a specialization of the Fractal component model [57].

PEAKFORCAST MIDDLEWARE auto-scaling system content two elements: a monitor (PeakForecast Client) and the scaling unit (PeakForecast Server).

PeakForecast Middleware monitoring system (PeakForecast Client), collects in real time the HTTP traffic (metrics) entering at the Load Balancer level at an appropriate granularity (eg per second, per minute), saving in the knowledge base and used by continuous machine learning process of traffic behavior.

PeakForecast Middleware scaling unit (PeakForecast Server) performs the scaling actions, e.g. add or remove a VM.

PEAKFORECAST MIDDLEWARE is implemented in Java and is therefore based on the use of the Java virtual machine (JVM).

To build an efficient and scalable solution, we built PEAKFORECAST MIDDLEWARE with the Akka event-driven middleware framework [2]. Akka is a toolkit for building highly concurrent, distributed, and resilient message-driven applications for Java. Akka is the implementation of the Actor Model on the JVM. The strength of Akka is its high performance: 50 million messages processed per second on a single machine, and small memory footprint; 2.7 million actors per GB of heap.

As a Knowledge base, we use MongoDB[3] to save the traffic http collected in real time, to exploit during the Learning Process.

3.3.2 PeakForecast Middleware Operations

In this section, we present the execution environment and operation.

The PeakForecast Middleware runtime environment is a typical web application with a 3-tiers architecture :

- **The Load Balancer tier :** receives all the incoming requests http from users and routes them to the application servers.
- **The Business-logic tier :** contains the application servers that execute the application logic.
- **The Storage tier :** refers to the database system.

The web application deployed on a IaaS infrastructure whose resources are Virtual Machines (VMs).

VMs may have the same or different resource assignment, but each VM has its own unique identifier (it could be the IP address). Final users will send requests to the load balancer. Each request will be assigned to a unique server. Each application server will be hosted in a different VM. We assume that servers can process several requests simultaneously, but note that the service time will increase depending on the number of concurrent requests. No that in a context in which we only scale horizontally the business-logic tier, we may use server and VM with the same meaning.

The initial situation is **(1)**. In this situation, we only have one VM in which we have deployed the application server (Business-logic tier). In frontend we have a VM in which we have deployed the Load Balancer. In the backend, a VM in which we have deployed the database (Storage tier).

In situation **(2)** the number of VMs increased. A new application server was added to the Business-logic tier, to cope with the increase in the number of users. Load balancing is performed between the two servers.

The Figure 9 illustrates this operation.

3.3.3 PeakForecast Middleware Properties

The main conjecture concerning the PeakForecast Middleware is to ensure each time by the following two properties :

1) **No Overload** : The IaaS system or infrastructure is always in operation regardless of the level of traffic.In fact our monitoring system collects in real time the HTTP traffic (metrics) entering at the Load Balancer level at an appropriate granularity , saving in the knowledge base and used by continuous machine learning process of traffic behavior.
2) **Auto-Scaling** : The quantity of active resources remains optimal compared to the level of traffic, if the traffic increases the resources are dynamically increased, if the traffic decreases the resources are dynamically reduced. In fact, our control mechanism orchestrates elasticity. This supervision is based on our MAPE-KL model of autonomous systems.

To ensure each time of its two properties, PeakForcast's supervisor coordinates the activation of the various components of the architecture in order to quickly arrive at an optimal preference decision. The function Monitor allows to collects traffic information per time interval in real time and save in Knowledge base. The number of requests collected, returns to function Analyse. The function analyse, analyzes this number of requests in , detecting Potential Traffic Surges, forecasting the Upcoming Traffic by using a forecasting model an estimating the number resources required to absorb the remaining traffic to come. Finally, the function Analyse returns this number of resources (VMs or Containers) to function Plan. The function Plan, based on the latency of VMs in use, determines the exact number of VM or container instances to deploy in order to absorb the traffic surge, verifies if the virtual web infrastructure is facing a Distributed Deny of Service (DDoS), and then returns number to function Execute. The function Execute takes care of providing the requested VMs or container instances, allowing the virtual web infrastructure to anticipate the traffic surge ahead. The function Learn, based on learning supervised as a machine learning technique, allows calculation (Learning Process) to part of a historic request

1. https://projects.ow2.org/view/frascati
2. https://akka.io
3. https://www.mongodb.com

(*e.g.*, 30 numbers of requests received) save in Knowledge base, to determine the best smoothing constant α and returns α to the function Analyse in order to increase and maintain the reliability of prediction model.

4 An Experimentation of the Model

In this section, we illustrate our approach on a case study that consists in absorbing a traffic surge that occurred on the Japanese version of Wikipedia the minutes before and after the tsunami on March 11th 2011. We present the experiment's data (sub-section 4.1), experiment's method (sub-section 4.2), the experiment's tools (sub-section 4.3), the result and discution (sub-section 4.4).

4.1 The experiment's data

On March 11, 2011, following a magnitude 9 earthquake, Japan experienced a tsunami [58] of historic magnitude, devastating entire cities and causing the Fukushima nuclear disaster.

From the Wikimedia Foundation websites [59], we obtained the http traffic that occurred on the Japanese version of Wikimedia the minutes before and after the March 11, 2011 tsunami.

Following the tsunami, many users requested the Japanese version of the mediawiki website. This significant, sudden and unpredictable build-up led to the total interruption of the service (i.e. inaccessible web page). This interruption of the service reflects a lack of elasticity in the system that has not been able to scale up. As a result, users did not have access to the service when they really needed it.

Regarding the preprocessing of http traffic data, the http traffic being collected in real time, we carried out an extraction of its data by one minute interval ([4]) from March 11, 2011.

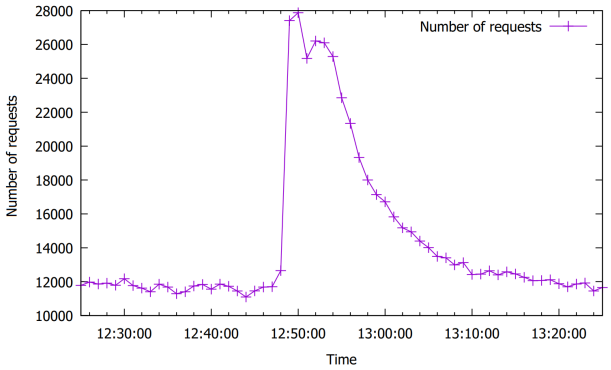

Fig. 10. Graphical representation of Wikipedia's http requests from 12:25:00 to 13:25:00 on March 11th 2011

In Figure 10, we can observe that, between 12:49 and 12:50, a traffic surge suddenly occurred because people constantly reported and consulted live information related to the disaster. One can observe that the intensity and the duration of such a surge are extreme compared to more traditional traffic loads. In particular, the short duration of this event makes it difficult to apply standard provisioning and deployment approaches to adjust the resources reactively.

4. https://github.com/fouomene/experiment-data-peakforecast

4.2 The experiment's tools

Concerning the tools of experimentation, the MediaWiki virtual web infrastructures (see Figure 11) have been set up.

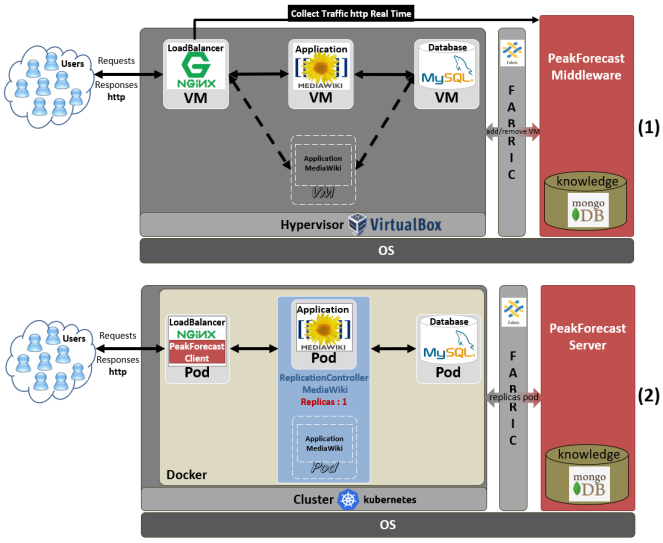

Fig. 11. MediaWiki Infrastructure Web and Peakforecast Middleware with **(1)** the Vitual Machines and **(2)** the Containers.

In its two MediaWiki virtual web infrastructures, PeakForecast Middleware monitors the incoming traffic, makes predictions on the volume of this traffic and adjust the size of the infrastructure according to these predictions. Adding more resources mainly concerns instances of *Virtual Machines* (VMs) or *Docker containers*[5] (Containers) hosting the MediaWiki Application, the business logic tier of the website, because they are usually considered as the bottleneck caused by traffic surges.

In particular, we use Fabric[6], a Python library for streamlining the use of SSH, for operating the VMs deployment or the replication containers.

Once instances of VMs are deployed, the load balancer (here Nginx[7]) is automatically reconfigured to consider the new instances made available.

8

- Case of the VMs (cf. Figure 11 (1))
  The physical machine uses Virtual Box[9] as an hypervisor and has the following characteristics: CORE i3 CPU 2.50GHz with 4 GB of RAM, and for each VM: CPU 1 GHz with 500 MB of RAM.
  A VM front-end hosts the load balancer Nginx[10] responsible for distributing incoming HTTP requests among the VMs hosting the MediaWiki Application [11] website (the business logic tier of the website).
  While another VM contain the MySQL server hosting the databases (the data layer of the website).

5. https://www.docker.com
6. http://fabfile.org
7. http://wiki.nginx.org
8. https://github.com/fouomene/experiment-data-peakforecast
9. https://www.virtualbox.org/wiki/Downloads
10. http://nginx.org/en/download.html
11. https://www.mediawiki.org/wiki/Download

- Case of the Containers (cf. Figure 11 (2)):
  The physical machine uses Kubernetes[12] for orchestrating, managing Docker containers and has the following characteristics: CORE i3 CPU 2.50GHz with 4 GB of RAM, and for each Container: CPU 1 GHz with 500 MB of RAM. The block "frontend-controller" which contains the container "*PeakForecastClient Nginx*", allows to monitor HTTP traffic, detect traffic surges and react accordingly.

For the purpose of this validation, adding resources concerns instances of VMs or Containers containing MEDIAWIKI APPLICATION, because they are considered as the bottleneck in traffic surges.

To perform our benchmark, we used the tool SIEGE[13]. SIEGE can stress a single URL with a user-defined number of simulated users, or it can read many URLs into memory and stress them simultaneously. The program reports the total number of hits recorded, bytes transferred, response time, concurrency, and return status. Siege supports HTTP/1.0 and 1.1 protocols, the GET and POST directives, cookies, transaction logging, and basic authentication.

4.3 The experiment's method

We will use the wikimedia data, to illustrate the two properties of PeakForecast: no overlow, and auto-scaling. First that the MEDIAWIKI Infrastructure Web (Figure 11) is always in operation regardless of the level of traffic, and secondly that The quantity of active VMs or containers remains optimal compared to the level of traffic, ie if the traffic increases the VMs or Containers contains MEDIAWIKI APPLICATION are dynamically increased, if the traffic decreases the VMs or Containers contains MEDIAWIKI APPLICATION are dynamically reduced.

In this case of Wikipedia, we will answer the question how to detect potential traffic surges, forecast the upcoming traffic and estimating the resource requirements.

Regarding the stages of the experiment ,

- The first stage of the experiment, before starting the virtual MEDIAWIKI web Infrastructure , the extracted experience data is saved in PEAKFORECAST MIDDLEWARE's MongoDB Knowledge base, then on startup the actor Monitor of PEAKFORECAST MIDDLEWARE, collects them as they go, respecting the time extraction interval.
- The second stage of the experimentation will consist of carrying out the simulation with the Peakforcast Middleware. At the start, we have a single VM containing an instance of APPLICATION MEDIAWIKI 1 and thus serving all HTTP requests sent by NGINX Load balancer. So to cope with this increase in load, PeakForecast Middleware should be able to detect potential traffic surges, forecasting the upcoming traffic by using a forecasting model, estimating the number resources required to absorb the remaining traffic to come, and auto–scaling resources by quickly automatically adding VMs to absorb traffic surges. The evolution of the CPU and RAM metrics of VMs will allow us to observe the behavior of the MEDIAWIKI Infrastructure Web (see Figure 11).

12. http://kubernetes.io
13. http://www.joedog.org/siege-home

4.4 Results and discussion

With data of wikipedia, one can observe that the intensity and the duration of such a surge are extreme compared to more traditional traffic loads. In particular, the short duration of this event makes it difficult to apply standard provisioning and deployment approaches to adjust the resources reactively. In this case, how to detect if we are facing a traffic surge caused by a Slashdot effect or not, predict the upcoming traffic?

4.4.1 *Wikipedia Detecting Potential Traffic Surges*

The intensity of such a traffic surge depends on a surge coefficient C_s that can be defined as:

$$C_s = \frac{\text{Number of requests at } P_n}{\text{Number of requests at } P_{n-1}}$$

The highest C_s, the more intense the traffic surge. This *surge coefficient* C_s, allows at actor Analyse to detect the start of the peak.

From this one minute interval extraction, we generate Figure 12, which illustrates the evolution of surge coefficient C_s between 12:28 and 13:22 on March 11th.

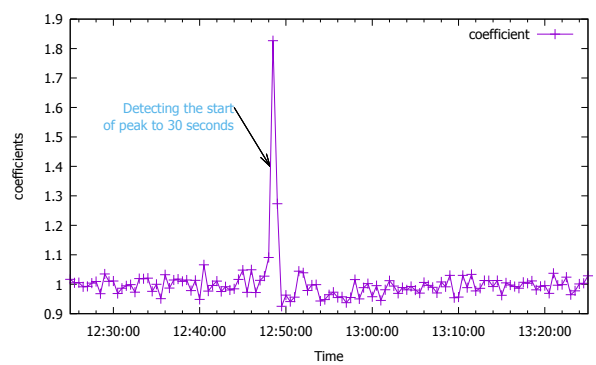

Fig. 12. Evolution of the surge coefficient on March 11th 2011.

The highest C_s , the more intense the traffic surge. Figure 12 illustrates the evolution of C_s between 12:28 and 13:22 on March 11th, and detecting a surge suspicion around 12:49. We note that if C_s is still substantially 1 and at period P, the curve is more than tenfold. This *surge coefficient* thus allows us to detect the start of the peak. In this case, every 30 seconds, when the *surge coefficient* is at least equal to 2, an alert is raised to notify a surge suspicion.

4.4.2 *Forecasting the Upcoming Traffic*

After detecting the traffic surge, we need to predict the traffic to be absorbed in the upcoming seconds. We applied the simple exponential smoothing method (see [56])on the requests throughput in a time interval enclosing the traffic surge (cf. Table 2). In Table 2, the first column are the intervals of 30 seconds from 12:46:30 to 12:53:30; for example #1=[12:46:30 - 12:47:00], #2=[12:47:30 - 12:47:30], #3=[12:47:30 - 12:48: 00]. The second column, the number of requests received in the time interval. Columns 3 to 5 are

the predictions of requests for each of its *smoothing constant*t values; For example in the column for α=0.9, at interval #3=[12:47:30 - 12:48:00], the query number is $5,890$, however the value that was predicted is $5,830$ and the prediction of the next interval is $5,884$. The table 3 are the squared of forecasting error per interval. Similarly for Table **??** and **??**.

Due to the recurring formula of simple exponential smoothing, we must choose a value from which forecasts can be applied. This value is of little importance if the series is long and in our case (cf. Table 2), we chose the first observation—*i.e.* the number of requests for the interval #1 $(5,981)$. Concerning the choice of the *smoothing constant* α, which directly affects the predictions, no direct mathematical method gives its optimum value. For this, it is obviously necessary to rely on data stored and therefore, in our case, based on requests history. Thus, we have compared three values of α—0.1, 0.4, and 0.9—to observe their effect on the predictions of our forecasting model.

Period Id	# of requests	Forecast α=0.4	Forecast α=0.1	Forecast α=0.9
1	5,981	5,981	5,981	5,981
2	5,813	5,981	5,981	5,981
3	5,890	5,914	5,964	5,830
4	6,051	5,904	5,957	5,884
5	6,600	5,963	5,966	6,034
6	12,056	6,218	6,030	6,543
7	15,349	8,553	6,632	11,505
8	14,198	11,271	7,504	14,965
9	13,674	12,442	8,173	14,275
10	12,870	12,935	8,723	13,734
11	12,306	12,909	9,138	12,956
12	12,851	12,668	9,455	12,371
13	13,351	12,741	9,794	12,803
14	13,065	12,985	10,150	13,296
15		13,017	10,441	13,088

TABLE 2
Traffic predictions per interval of 30 seconds from 12:46:30 to 12:53:30 on March 11th 2011.

Id	# of requests	Residual α=0.4	Residual α=0.1	Residual α=0.9
1	5,981			
2	5,813	28,224	28,224	28,224
3	5,890	566	5,505	3,624
4	6,051	21,527	8,877	27,896
5	6,600	405,810	401,700	320,019
6	12,056	34,084,803	36,317,716	30,388,430
7	15,349	46,184,685	75,982,191	14,778,312
8	14,198	8,564,747	44,810,958	587,636
9	13,674	1,517,665	30,257,578	360,789
10	12,870	4204	17,194,457	746,610
11	12,306	363,492	10,035,958	423,029
12	12,851	33,583	11,533,918	230,361
13	13,351	372,045	12,649,019	300,300
14	13,065	6,396	8,496,592	53,454
$\sum e_i^2$		9.15×10^7	2.47×10^8	4.82×10^7

TABLE 3
Residuals $e_i^2 = (y_i - \hat{y}_i)^2$ of Table 2.

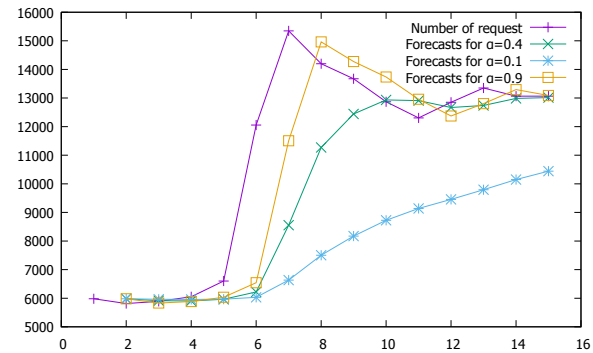

Fig. 13. Traffic predictions from 12:46:30 to 12:53:30 on March 11th 2011.

From Table 3, we can observe that the *smoothing constant* minimizing $\sum e_i^2$ and provides predictions with an acceptable precision—*i.e.*, which is very close to reality—is $\alpha = 0.9$. This can be acknowledged in Figure 13.

With this *constant smoothing* of 0.9, when we detect a traffic surge at the interval #6 (cf. Table 2), we predict an increase of $11,504$ requests during the next period. Adding this estimated traffic to previous requests, we obtain a forecast traffic of $23,560$ requests from the 30th second of the minute marking the traffic surge, as shown in Figure 10.

This prediction is close to the peak of real traffic, which reached $27,405$ requests per minute (adding the interval number 6 and 7 of Table 2), with an average of 457 requests per second.

4.4.3 Estimating the resource requirements

While the duration of a traffic surge cannot be predicted, another key constant to be considered is the time required to bring a computing resource to be ready to process request.

Case of *virtual machines* (VMs): This includes the time to provision a virtual appliance, the time to install an application (*e.g.*, MEDIAWIKI takes 10 seconds on average to install), a server or middleware platform (*e.g.*, MONGODB takes 11 seconds on average to install) on a virtual machine. By taking into account this delay and the maximum throughput supported by the considered components, we can estimate the number of virtual appliances and application stacks required to absorb the traffic surge along the upcoming period. In particular, we use the following formula where the number of virtual appliances to deploy N is estimated as:

$$N = \frac{S_{t+1} - Y_t}{X}$$

Case of *Docker containers* (Containers): We use the following formula where the number of virtual appliances to deploy N is estimated as:

$$N = \frac{S_{t+1}}{X}$$

Where S_{t+1} is the *estimated traffic* at time $t+1$, Y_t is the *monitored traffic* at time t, X is the *maximum throughput* supported by a given application stack, about 600 requests per second in our case study (this value is obtained from benchmarks). The functions respectively calculating the number of resources to provision for VMs and containers

are different, because in containers the number of resources calculate includes resources in use. We do it in relation to the Kubernate[14] command which allows us to scale the number of containers(For example, with command *$ Kubectl scale —-replicas = 3* , if initially there are 2 containers then Kubernate will only add 1 container).

4.4.4 PeakForecast Middleware simulation result with Wikipedia data

The result of the experimentation steps given in the experimation design part gave us this:

- In Figure 14(a), we have a single VM containing an instance of MEDIAWIKI APLICATION 1 and thus serving all HTTP requests sent by NGINX load balancer. PEAKFORECAST MIDDLEWARE quickly detects that the infrastructure is facing a traffic surge, predicts the upcoming traffic, estimates the number of VMs required to absorb the traffic surge (2 VMs) and starts the process of provisioning these VMs.
- In Figure 14(b), PEAKFORECAST MIDDLEWARE installs MEDIAWIKI APPLICATION on VMs `VM 2 & VM 3` (in approximately 10 seconds), then add the VMs in the configuration file of the NGINX load balancer to make them visible to NGINX.
- Figure 14(c) reports treatments, such as requests sent by VMs `Mediawiki VM 2 & MediaWiki VM 3`, which explains the evolution of CPU and RAM in VM `VM 1`.
- In Figure 14(d), PEAKFORECAST MIDDLEWARE removes the VMs `MediaWiki VM 2 & MediaWiki VM 3` once the traffic surge has been absorbed.

We also have a single container containing MEDIAWIKI APPLICATION and thus serving all HTTP requests sent by frontend-controller.

- In Figure 15(b) PEAKFORECAST MIDDLEWARE quickly detects that the infrastructure is facing a traffic surge, predicts the upcoming traffic, estimates the number of Containers required to absorb the traffic surge (2 Containers: `MediaWiki container 2 & 3`) and starts the process of provisioning these Containers by replication mediawiki-controller (in approximately 5 seconds).
- Figure 15(c) reports treatments, such as requests sent by Containers `MediaWiki Container 2 & 3`, which explains the evolution of CPU and RAM in container `MediaWiki Application 1`.
- In Figure 15(d), PEAKFORECAST MIDDLEWARE removes the Container `MediaWiki Container 2 & 3` once the traffic surge has been absorbed.

5 CONCLUSION AND PERSPECTIVES

In this paper, to response the following research questions: Is it possible to anticipate the effects of a traffic surge? If so, is it possible to protect the targeted system from unavailability?

To address this challenge, we propose an autonomic elasticity management approach in the cloud. The interest of this approach lies, on one hand, on the process of continuous learning of traffic behavior, whose knowledge improves the detection of future traffic surges, and on the other hand, in a simultaneous allocation mecanism of resources during the process of auto-scaling cloud resources, with the aim of absorbing the detected traffic surges.

In order to increase and maintain the reliability of the forecasting model implemented for the Analysis phase (Analyze), a machine learning phase (Learn) has been incorporated to extend the control loop MAPE-K to a MAPE-KL (Monitor, Analyze, Plan, Execute, Knowledge and Learn) loop. A supervised machine learning method is used with the implementation of a regression algorithm, in order to permanently fine tune the traffic forecasting model.

The initialization time of cloud resources is not negligible, it sometimes induces a lack of responsiveness to changes in system states. Indeed, the time required to start an instance and all the associated services can take several minutes preventing the ideal of "just-in-time sizing" from being reached. We therefore use the actor model of concurrent programming in order to ensure simultaneous auto-scaling of resources. In this model the parallelism of execution is expressed explicitly.

Our mix approach consists, from a trace of queries received in the last seconds, minutes or hours of : *i)* Detecting potential traffic surges, *ii)* Forecasting the upcoming traffic and, *iii)* Estimating the number resources required to absorb the remaining traffic to come, *iv)* Auto–scaling resources by quickly automatically adding resources to absorb traffic surges. This approach offers benefits for multi–tier applications that are already implemented using multiple VMs by improving the resource utilization between them as application demands vary.

We have evaluated approach using MediaWiki and web application traffic traces that were collected on the Japanese version of Wikipedia some days before and after the tsunami on March 11th, 2011, in an interval where the traffic surge has been observed. The experimental results confirm that a prototype elastic middleware solution(PEAKFORECAST MIDDLEWARE), can provide instantaneous elasticity of resources during traffic surges.

As for future work, we planned: *i)* To give PEAKFORECAST MIDDLEWARE the possibility of quickly changing some VM parameters (Vertical Scaling e.g. memory, CPU) to delay the peak of traffic before considering the deployment of additional VMs or Containers. *ii)* In real commercial scenarios, allocating extra VMs or Containers will have to be paid for a whole hour. Thus, we should allow PEAKFORECAST MIDDLEWARE, based on predictions, to consider the expected duration of the traffic surge in order to adjust the minimum number of VMs or Containers that would maintain an acceptable QoS and minimize wasting of resources.

14. http://kubernetes.io

REFERENCES

[1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," Computer, vol.36, no.1, p.41-50, 2003

[2] Tania Lorido-Botrean, Josee Miguel-Alonso, Josee A. Lozano, "Auto-scaling Techniques for Elastic Applications in Cloud Environments, " Technical Report September 5, 2012.

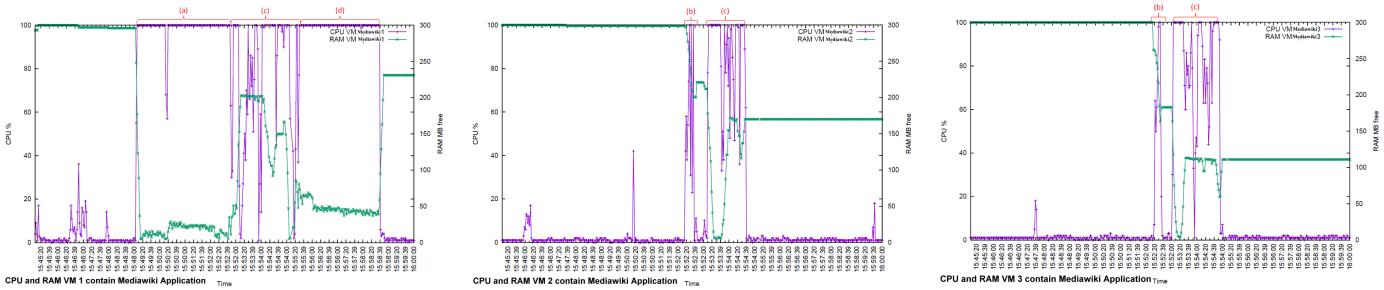

Fig. 14. Benchmark 1 MEDIAWIKI Infrastructure Web with VMs

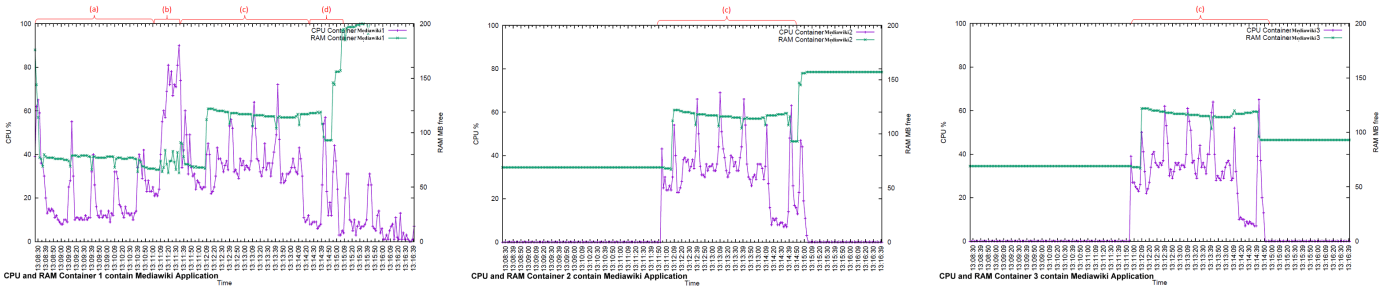

Fig. 15. Benchmark 2 MEDIAWIKI Infrastructure Web with Containers

[3] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling Web Applications in Clouds: A Taxonomy and Survey," arXiv preprint-arXiv:1609.09224, 2016.

[4] M. Mohamed, M. Amziani, D. BelaA'rd, S. Tata, and T. Melliti, "An autonomic approach to manage elasticity of business processes in the Cloud," Future Generation Computer Systems, 2014.

[5] M. Ghobaei-Arani, S. Jabbehdari, and M. A. Pourmina, "An autonomic approach for resource provisioning of cloud services," Cluster Computing, pp. 1-20, 2016.

[6] R. Weingartner, G. B. Brascher, and C. B. Westphall, "Cloud resource management : A survey on forecasting and profiling models," Journal of Network and Computer Applications, vol. 47, pp. 99-106, 2015.

[7] William LeFebvre. CNN.com: Facing A World Crisis. Invited Talk, USENIX ATC 2002.

[8] Jim Hu and Greg Sandoval. Web acts as hub for info on attacks. CNET news, September 2001.

[9] Lisa A. Wald and Stan Schwarz. The 1999 southern california seismic network bulletin. Seismological Research Letters, 71:401-422, July 2000.

[10] Stephen Adler. The Slashdot Effect: An Analysis of Three Internet Publications. http://ssadler.phy.bnl.gov/adler/SDE/SlashDotEffect.html.

[11] Josh Constine. Walmarts black friday disaster: Website crippled, violence in stores. http://techcrunch.com/2011/11/25/walmart-black-friday, November 2011.

[12] Kathleen Ohlson. Victorias secret knows ads, not the web. Computer World, February 1999

[13] Martin Arlitt and Tai Jin. Workload characterization of the 1998 world cup web site. IEEE Network, 1999.

[14] Peter Pachal. Amazon apologizes for cloud outage, issues credit to customers. PCMag, April 2011.

[15] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. DRAM errors in the wild: a large-scale field study. In SIGMETRICS 2009, Seattle, WA, USA.

[16] Interview de Todd Papaioannou par Julie Bort. http://www.networkworld.com/article/2179359/cloud-computing/yahoo-builds-ultimate-private-cloud.html, July 2011.

[17] "Animoto's Facebook Scale-Up," [Online]. Available: http://tinyurl.com/qdk7om4.

[18] Pooyan Jamshidi, Aakash Ahmad, and Claus Pahl. Autonomic resource provisioning for cloud-based software. In Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pages 95-104. ACM, Pages 95-104 , 2014.

[19] G. Linden, "Make Data Useful," Amazon, 2009.

[20] Joao Loff et Joao Garcia. Vadara : Predictive Elasticity for Cloud Applications. In Proceedings of the 2014 IEEE 6th International Conference on Cloud Computing Technology and Science, CLOUDCOM14, pages 541-546 Washington, DC, USA, 2014. IEEE Computer Society.

[21] H. Nguyen, Z. Shen, X. Gu, S. Subbiah, and J. Wilkes. Agile : Elastic distributed resource scaling for infrastructure-as-a-service. In Proc. of the USENIX International Conference on Automated Computing (ICAC13). San Jose, CA, 2013.

[22] N. Roy, A. Dubey, A. Gokhale, Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting, Proceedings of the 2011 IEEE 4th International Conference on Cloud Computing, pp. 500-507, 2011.

[23] Z. Gong, J. Wilkes, and X. Gu, Press: Predictive elastic resource scaling for cloud systems, in 2010 International Conference on Network and Service Management. IEEE, 10 2010, pp. 9-16.

[24] A. Krioukov, P. Mohan, S. Alspaugh, L. Keys, D. Culler, and R. Katz. Napsac: Design and implementation of a power-proportional web cluster, In Green Networking 2010, New Delhi, India, p. 15-22.

[25] T. Horvath and K. Skadron: Multi-mode energy management for multi-tier server clusters, In PACT 2008, Toronto, ON, Canada, p. 270-279.

[26] R. Han, L. Guo, M. M. Ghanem, Y. Guo, Lightweight Resource Scaling for Cloud Applications, Proceedings of the 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, pp. 644-651, 2012

[27] J.C.B. Leite, D.M. Kusic, and D. Mosse : Stochastic approximation control of power and tardiness in a three-tier web-hosting cluster, In ICAC 2010, Washington, DC, USA : 41-50.

[28] R. Nathuji, A. Kansal, and A. Ghaffarkhah. : Q-clouds: Managing performance interference effects for qos-aware clouds, In EuroSys 2010, Paris, France, p. 237-250.

[29] P. Padala, K.-Y. Hou, K.G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal, and A. Merchant : Automated control of multiple virtualized resources, In EuroSys 2009, Nuremberg, Germany, p. 13-26.

[30] B. Urgaonkar, P. Shenoy, A. Chandra, P. Goyal and T. Wood, Agile dynamic provisioning of multi-tier internet applications, ACM TAAS, 2008.

[31] Kathryn Bean Laura R. Moore and Tariq Ellahi. A coordinated reactive and predictive approach to cloud elasticity. CLOUD COMPUTING 2013 : The Fourth International Conference on Cloud Computing, GRIDs, and Virtualization, 2013

[32] A. Ali-Eldin, J. Tordsson and E. Elmroth, An adaptive hybrid

elasticity controller for cloud infrastructures in Network Operations and Management Symposium (NOMS), IEEE, 2012, pp. 204-212

[33] Sourav Dutta, Sankalp Gera, Akshat Verma, and Balaji Viswanathan. Smartscale : Automatic application scaling in enterprise clouds. In Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on, pages 221-228. IEEE, 2012

[34] A. Gandhi, Y. Chen, D. Gmach, M. Arlitt, and M. Marwah: Minimizing data center sla violations and power consumption via hybrid resource provisioning, In IGCC 2011, Orlando, FL, USA, p. 1-8.

[35] D. Gmach, S. Krompass, A. Scholz, M. Wimmer, and A. Kemper: Adaptive quality of service management for enterprise services, ACMTrans. Web, 2(1):1-46, 2008.

[36] B. Urgaonkar and A. Chandra : Dynamic provisioning of multi-tier internet applications, In ICAC 2005, Washington, DC, USA, p.217-228.

[37] Paul-Jean CAGNARD, PARCEL-2 : Un modèle de programmation parallèle, cellulaire, hiérarchique, Thèse de doctorat à L'école Polytechnique Fédérale de Lausanne, 2000. URL : http://pierre.kuonen.home.hefr.ch/grip/pdf/these-a4.pdf

[38] S. B. Kotsiantis, "Supervised Machine Learning: A Review of Classification Techniques", Proceedings of the 2007 Conference on Emerging Artificial Intelligence Applications in Computer Engineering: Real Word AI Systems with Applications in eHealth, HCI, Information Retrieval and Pervasive Technologies, 2007, pages 3-24, http://dl.acm.org/citation.cfm?id=1566770.1566773

[39] Ming Mao and Marty Humphrey. A performance study on the vm startup time in the cloud. In Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on, pages 423–430. IEEE, 2012. 32

[40] Loïc Letondeur, Xavier Etchevers, Thierry Coupaye, Fabienne Boyer et Noël De Palma. Architectural Model and Planification Algorithm for the Self-Management of Elastic Cloud Applications. In Proceedings of the 2014 International Conference on Cloud and Autonomic Computing, ICCAC'14, pages 172–179, Washington, DC, USA, 2014. IEEE Computer Society.

[41] Ying Zhang, Gang Huang, Xuanzhe Liu, and Hong Mei. Integrating resource consumption and allocation for infrastructure resources on-demand. In Cloud Computing (CLOUD), 2010 IEEE 3rd International Conference on, pages 75–82. IEEE, 2010.

[42] Nilesh N Karnik, Jerry M Mendel, and Qilian Liang. Type-2 fuzzy logic systems. Fuzzy Systems, IEEE Transactions on, 7(6) :643–658, 1999.

[43] Paul Horn. Autonomic Computing : IBM's Perspective on the State of Information Technology. Rapport technique, IBM, 2001

[44] Mohammed El Amine MATOUGUI, Vers un environnement pour le déploiement logiciel autonomique, Thèse de doctorat de TELECOM Sud Paris, le 21 novembre 2013. URL : https://tel.archives-ouvertes.fr/tel-00926023v2/document

[45] Brandic Ivona, Towards self-manageable cloud services, In Computer Software and Applications Conference, 2009. COMPSAC'09. 33rd Annual IEEE International, volume 2, pages 128–133. IEEE, 2009. URL : http://ieeexplore.ieee.org/xpl/articleDetails.jsp?arnumber=5254138

[46] Maurer Micheal, Breskovic Ivan, Emeakaroha Vincent C et Brandic Ivona, Revealing the MAPE loop for the autonomic management of Cloud infrastructures, In IEEE Symposium on Computers and Communications (ISCC), pages 147–152. IEEE, 2011. URL: http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=5984008

[47] Youssef LATROUS, Gestion dynamique d'une architecture cellulaire massivement parallèle, Thèse de doctorat de l'Institut National Polytechnique de Grenoble, le 12 Janvier 1995. URL : https://tel.archives-ouvertes.fr/tel-00005051/document

[48] G. Agha, ACTORS: a model of concurrent computation in distributed systems, MIT Press, 1996. URL: https://www.cypherpunks.to/erights/history/actors/AITR-844.pdf

[49] Carl Hewitt et Henry Baker Jr., Actors and Continuous Functionals, IFIP Working Conference on Formal Description of Programming Concepts, St Andrews, Canada, 1977. URL: https://www.cypherpunks.to/erights/history/actors/AITR-844.pdf

[50] Marino Jim et Rowley Michael, Understanding SCA (Service Component Architecture). Addison-Wesley Professional, 2010. URL: http://dl.acm.org/citation.cfm?id=1594903

[51] Seinturier Lionel, Merle Philippe, Fournier Damien, Dolet Nicolas,Schiavoni Valerio et Stefani J-B, Reconfigurable SCA applications with the FraSCAti platform, In international Conference on Service Computing, SCC'09., pages 268-275. IEEE, 2009. URL : https://hal.inria.fr/inria-00397856/file/frascati.pdf

[52] Seinturier Lionel, Merle Philippe, Rouvoy Romain, Romero Daniel, Schiavoni Valerio et Stefani Jean-Bernard, A component-based middleware platform for reconfigurable service-oriented architectures, Software : Pratice and Experience, 42(5) :559-583, 2012. URL: https://hal.inria.fr/inria-00567442/document

[53] Papazoglou Mike. P et Tranverso Paolo, Dustdar Schahram, AET Leymann Frank, Service-oriented computing: State of the art and research chalenges, Computer, 40(11) :38-45, 2007. URL: http://infolab.uvt.nl/staff/mikep/publications/PapazoglouTDL07.pdf

[54] Beisiegel Michael, Booz Dave, Colyer Adrian, Hildebrand Hal, Marino Jim et Tam Ken, SCA Service Component Architecture, Mars 2007. URL: http://xml.coverpages.org/SCA-AssemblyModel-V100.pdf

[55] Fawaz Paraiso, soCloud : une plateforme multi-nuages distribuée pour la conception, le déploiement et l'exécution d'applications distribuées à large échelle, Thèse de doctorat de l'Université de Lille 1, le 18 Juin 2014. URL: https://tel.archives-ouvertes.fr/tel-01009918v2/document

[56] R.G. Brown, R.F. Meyer: The fundamental theory of exponential smoothing, Operations Research, 9, 1961, p. 673–685

[57] E. Bruneton, T. Coupaye, M. Leclercq, V. Quema, and J.–B. Stefani: The Fractal Component Model and its Support in Java, Software Practice and Experience (SPE), 36(11–12):1257–1284, 2006.

[58] https://en.wikipedia.org/wiki/2011_Tohoku_earthquake_and_tsunami

[59] Website traffic of wikipedia https://dumps.wikimedia.org/

[60] E. Caron, L. Rodero–Merino, F. Desprez and A. Muresan, Autoscaling load balancing and monitoring in commercial and open-source clouds, RR-7857, INRIA 2012, pp.27.

[61] Pooyan Jamshidi, Aakash Ahmad, and Claus Pahl. Autonomic resource provisioning for cloud–based software. In Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self–Managing Systems, pages 95–104. ACM, Pages 95-104 , 2014.

[62] A. Computing, "An architectural blueprint for autonomic computing," IBM White Paper, vol. 31, 2006.

[63] A. Gambi, G. Toffetti and M. Pezze, Assurance of self–adaptive controllers for the cloud, in Assurances for Self-Adaptive Systems, pp 311-339, 2013.

[64] Amazon web services, 2017, last visited: 2017/04/08. [Online]. Available: https://aws.amazon.com

[65] RightScale 2017, last visited: 2017/04/08. [Online]. Available: http://www.rightscale.com

[66] Microsoft Azure 2017, last visited: 2017/08/25. [Online]. Available: https://azure.microsoft.com.

[67] RightScale, Set up Autoscaling using Voting Tags, last visited: 2017/08/25 [Online]. Available: http://support.rightscale.com/12-Guides/Dashboard_Users_Guide/Manage/Arrays/Actions/Set_up_Autoscaling .

[68] Thraka, Andy . How to scale a cloud service, Azure 2015. url: http://azure.microsoft.com/en-gb/documentation/articles/cloud-services-how-to-scale .

[69] Jonathan Kupferman, Jeff Silverman, Patricio Jara, and Jeff Browne. Scaling into the cloud. Technical report, University of California, Santa Barbara; CS270 - Advanced Operating Systems, 2009.

[70] SPICER, James. CloudPlan: An Elastic Management System for Cloud Planning. Discovery, Invention & Application, [S.l.], june 2016. Available at: https://computing.derby.ac.uk/ojs/index.php/da/article/view/101. Date accessed: 25 aug. 2017.

[71] D. Gmach, J. Rolia, L. Cherkasova, and A. Kemper. Capacity management and demand prediction for next generation data centers. In International Conference on Web Services, 2007.

[72] G. Chen and et al., Energy-aware server provisioning and load dispatching for connection-intensive internet services, in Proc. NSDI, 2008.

[73] P. Saripalli, G. Kiran, R. R. Shankar, H. Narware, and N. Bindal, Load prediction and hot spot detection models for autonomic cloud computing, 2011 Fourth IEEE International Conference on Utility and Cloud Computing, pp. 397–402, 12 2011.

[74] X. Zhu and et al., 1000 Islands: Integrated capacity and workload management for the next generation data center, in Proc. ICAC, Jun. 2008

[75] E. Kalyvianaki, T. Charalambous, and S. Hand, Self-adaptive and self-configured CPU resource provisioning for virtualized servers using Kalman filters, in Proc. ICAC, 2009.
[76] P. Padala and et al., Adaptive control of virtualized resources in utility computing environments, in Proc. Eurosys, 2007.
[77] J. Rao, X. Bu, C.-Z. Xu, L. Wang, and G. Yin, VCONF: A reinforcement learning approach to virtual machines auto-configuration, in Proc. ICAC, 2009.
[78] N. Huber, F. Brosig, and S. Kounev. Model-based self-adaptive resource allocation in virtualized environments. In Software Engineering for Adaptive and SelfManaging Systems, 2011.
[79] F. Brosig, N. Huber, and S. Kounev. Automated extraction of architecture-level performance models of distributed component-based systems. In Automated Software Engineering, 2011.
[80] H. Nguyen, Z. Shen, X. Gu, S. Subbiah, and J. Wilkes. Agile: Elastic distributed resource scaling for infrastructure-as-a-service. In Proc. of the USENIX International Conference on Automated Computing (ICAC13). San Jose, CA, 2013.
[81] L. M. Vaquero, L. Rodero-Merino and R. Buyya, Dynamically scaling applications in the cloud, ACM SIGCOMM Computer Communication Review,Pages 45–52, 2011.
[82] B. Urgaonkar and P. Shenoy: Cataclysm: Scalable overload policing for internet applications, Journal of Network and Computer Applications, 31(4):891–920, 2008.
[83] A. Adya, W.J. Bolosky, R. Chaiken, J.R. Douceur, J. Howell, and J. Lorch: Load management in a large-scale decentralized file system, MSR-TR, 2004–60, July 2004.
[84] T. Voigt, R. Tewari, D. Freimuth, and A. Mehra: Kernel mechanisms for service differentiation in overloaded web servers. In USENIX ATC 2001, Boston, MA, USA, p. 189–202.
[85] L. Cherkasova and P. Phaal: Session-based admission control: A mechanism for peak load management of commercial web sites, IEEE Trans. Comput., 51, June 2002, p. 669–685.
[86] H. C. Lim, S. Babu, and J. S. Chase: Automated control for elastic storage, in ICAC10. New York,NY, USA: ACM, 2010, p. 19–24.
[87] B. Trushkowsky, P. Bodik, A. Fox, M.J. Franklin, M.I. Jordan, and D.A. Patterson: The scads director: scaling a distributed storage system under stringent performance requirements, In FAST 2011, San Jose, CA, USA, p. 163–176.
scale/.
scale/.